

VS

Vehicle Service & Maintenance System

Database Design Documentation

7

Tables

18

Procedures

5

Functions

4

Triggers

Table of Contents

1	Project Overview	3
2	Database Architecture	3
3	Table Designs & Relationships	4
3.1	Users Table	4
3.2	Customers Table	4
3.3	Vehicles Table	5
3.4	ServiceSchedule Table	5
3.5	ServiceDetails Table	6
3.6	Complaints Table	6
3.7	AuditLog Table	7
4	Relationships & Foreign Keys	7
5	Stored Procedures	8
6	Functions	9
7	Triggers	9
8	Business Logic Summary	10

1 Project Overview

The **Vehicle Service & Maintenance System** is a SQL Server database designed to manage all aspects of a vehicle service centre. It covers user authentication, customer and vehicle records, service scheduling, service detail tracking, complaint handling, and full audit logging — everything needed to run a professional auto-service business.

Module	What It Does
User Management	Controls who can log in and what they can do (Admin or Staff).
Customer Management	Stores customer contact details and service history.
Vehicle Management	Links every vehicle to its owner and tracks its details.
Service Scheduling	Books vehicles in for service on specific dates.
Service Details	Records exactly what was done, parts used, and cost.
Complaint System	Logs customer complaints, assigns them to staff, and tracks resolution.
Audit Log	Automatically records every important change in the database.

2 Database Architecture

The database follows a **relational model** with clearly defined Primary Keys (PK) and Foreign Keys (FK). Every table is connected logically so data is never duplicated — instead, tables reference each other through IDs.

Component	Count	Purpose
Tables	7	Store all data in organised, structured form
Stored Procedures	18	Pre-written SQL to add, update, delete, and report data
Functions	5	Return calculated values (totals, counts, labels)
Triggers	4	Automatic actions that fire when data changes
Constraints	30+	Rules that protect data quality (NOT NULL, UNIQUE, CHECK)

3 3 Table Designs & Relationships

3.1 Users Table

Stores all system users — both Admins and Staff. Every login action uses this table. Passwords are stored as hashed values, never plain text.

Column Name	Data Type	Key	Constraint / Notes
UserID	INT	PK	Auto-incremented unique identifier
Username	VARCHAR(50)		NOT NULL, UNIQUE — no two users share a username
PasswordHash	VARCHAR(256)		NOT NULL — hashed password for security
FullName	VARCHAR(100)		NOT NULL — display name of the user
Role	VARCHAR(20)		CHECK: must be 'Admin' or 'Staff'
Email	VARCHAR(100)		UNIQUE — optional contact email
Phone	VARCHAR(20)		Optional phone number
IsActive	BIT		DEFAULT 1 — soft delete flag (0 = deactivated)
CreatedAt	DATETIME		DEFAULT GETDATE() — auto-set on creation

Note: Users are never hard-deleted. Setting IsActive=0 disables the account safely.

3.2 Customers Table

Holds all customer personal information. Each customer can own multiple vehicles. The system tracks when the customer record was created.

Column Name	Data Type	Key	Constraint / Notes
CustomerID	INT	PK	Auto-incremented unique identifier
FullName	VARCHAR(100)		NOT NULL — customer's full name
Email	VARCHAR(100)		UNIQUE — optional email address
Phone	VARCHAR(20)		NOT NULL — primary contact number
Address	VARCHAR(255)		Optional home/business address
CreatedAt	DATETIME		DEFAULT GETDATE()

3.3 Vehicles Table

Each vehicle belongs to one customer (FK → Customers). A customer may have many vehicles. The license plate must be unique across the entire system.

Column Name	Data Type	Key	Constraint / Notes
VehicleID	INT	PK	Auto-incremented unique identifier

Column Name	Data Type	Key	Constraint / Notes
CustomerID	INT	FK	References Customers(CustomerID) — owner
Make	VARCHAR(50)		NOT NULL — brand e.g. Toyota
Model	VARCHAR(50)		NOT NULL — model e.g. Corolla
Year	INT		CHECK: between 1900 and 2100
LicensePlate	VARCHAR(20)		NOT NULL, UNIQUE
VIN	VARCHAR(50)		UNIQUE — vehicle identification number
Color	VARCHAR(30)		Optional colour
Mileage	INT		CHECK: >= 0
CreatedAt	DATETIME		DEFAULT GETDATE()

3.4 ServiceSchedule Table

This is the booking table. When a service is scheduled, both the vehicle and the customer are linked. The status moves from 'Scheduled' → 'In Progress' → 'Completed' automatically via triggers.

Column Name	Data Type	Key	Constraint / Notes
ScheduleID	INT	PK	Auto-incremented unique identifier
VehicleID	INT	FK	References Vehicles(VehicleID)
CustomerID	INT	FK	References Customers(CustomerID)
ScheduledDate	DATE		NOT NULL — date of the service appointment
ServiceType	VARCHAR(100)		NOT NULL — e.g. Oil Change, Full Service
Status	VARCHAR(20)		CHECK: Scheduled / In Progress / Completed / Cancelled
Notes	VARCHAR(500)		Optional notes from staff
CreatedBy	INT	FK	References Users(UserID) — who made the booking
CreatedAt	DATETIME		DEFAULT GETDATE()

3.5 ServiceDetails Table

Records the actual work performed during a service. Linked to a schedule (FK → ServiceSchedule) and a vehicle. Tracks cost, parts used, and completion time.

Column Name	Data Type	Key	Constraint / Notes
ServiceID	INT	PK	Auto-incremented unique identifier
ScheduleID	INT	FK	References ServiceSchedule(ScheduleID)
VehicleID	INT	FK	References Vehicles(VehicleID)
TechnicianID	INT	FK	References Users(UserID) — staff who did the work
ServiceDate	DATE		DEFAULT today's date
Description	VARCHAR(1000)		NOT NULL — full description of work done
Status	VARCHAR(20)		CHECK: Pending / In Progress / Completed
ServiceCost	DECIMAL(10,2)		CHECK: >= 0, DEFAULT 0
PartsUsed	VARCHAR(500)		List of parts/items used
CompletedAt	DATETIME		Auto-set when status becomes Completed
CreatedAt	DATETIME		DEFAULT GETDATE()

3.6 Complaints Table

Supports the Complaint Management System. Customers submit complaints which are assigned to staff and resolved. The full lifecycle is tracked: Open → In Progress → Resolved → Closed.

Column Name	Data Type	Key	Constraint / Notes
ComplaintID	INT	PK	Auto-incremented unique identifier
CustomerID	INT	FK	References Customers(CustomerID)
VehicleID	INT	FK	Optional — if complaint is about a vehicle
Subject	VARCHAR(200)		NOT NULL — short complaint title
Description	VARCHAR(2000)		NOT NULL — full complaint details
Status	VARCHAR(20)		CHECK: Open / In Progress / Resolved / Closed
AssignedTo	INT	FK	References Users(UserID) — assigned staff
ResolutionNotes	VARCHAR(2000)		Staff notes on how it was resolved
CreatedAt	DATETIME		DEFAULT GETDATE()
ResolvedAt	DATETIME		Set when complaint is resolved

3.7 AuditLog Table

Automatically populated by Triggers — never manually. Every INSERT, UPDATE, and DELETE on key tables creates an audit entry. Useful for tracking who changed what and when.

Column Name	Data Type	Key	Constraint / Notes
LogID	INT	PK	Auto-incremented
TableName	VARCHAR(50)		Name of the table that was changed
Action	VARCHAR(10)		CHECK: INSERT / UPDATE / DELETE
RecordID	INT		ID of the affected record
ChangedBy	VARCHAR(50)		Username or process that made the change
ChangedAt	DATETIME		DEFAULT GETDATE()
OldValues	VARCHAR(MAX)		Snapshot of data before the change
NewValues	VARCHAR(MAX)		Snapshot of data after the change

Note: This table is append-only. No updates or deletes should ever be made to it.

4 Relationships & Foreign Keys

The diagram below shows how all 7 tables are connected. Lines represent Foreign Key relationships. Each relationship enforces **referential integrity** — you cannot add a vehicle for a customer that doesn't exist, for example.

Parent Table	Child Table	FK Column	Meaning
Customers	Vehicles	CustomerID	A customer owns one or many vehicles
Customers	ServiceSchedule	CustomerID	A customer can have many bookings
Vehicles	ServiceSchedule	VehicleID	A vehicle can be booked many times
Users	ServiceSchedule	CreatedBy	A user creates the booking
ServiceSchedule	ServiceDetails	ScheduleID	Details are linked to a schedule
Vehicles	ServiceDetails	VehicleID	Details record which vehicle was serviced
Users	ServiceDetails	TechnicianID	Which staff member did the work
Customers	Complaints	CustomerID	A customer raises a complaint
Vehicles	Complaints	VehicleID	Optional: complaint about a vehicle
Users	Complaints	AssignedTo	Staff assigned to handle complaint

5 5 Stored Procedures

Stored Procedures are pre-written SQL blocks saved inside the database. Instead of writing raw SQL from your application, you call a procedure by name. This makes code reusable, secure, and easier to maintain.

User Management

Procedure Name	What It Does
<code>sp_AddUser</code>	Creates a new user account. Checks for duplicate username first.
<code>sp_UpdateUser</code>	Updates user info (name, role, email, active status).
<code>sp_DeleteUser</code>	Soft-deletes a user by setting IsActive = 0.

Customer Management

Procedure Name	What It Does
<code>sp_AddCustomer</code>	Adds a new customer. Returns the new CustomerID.
<code>sp_UpdateCustomer</code>	Updates a customer's contact details.
<code>sp_DeleteCustomer</code>	Deletes customer only if they have no vehicles.

Vehicle Management

Procedure Name	What It Does
<code>sp_AddVehicle</code>	Registers a new vehicle for a customer.
<code>sp_UpdateVehicle</code>	Updates vehicle details (make, model, plate, mileage, etc.).
<code>sp_DeleteVehicle</code>	Deletes vehicle only if it has no active service schedules.

Service Scheduling

Procedure Name	What It Does
<code>sp_ScheduleService</code>	Books a vehicle in for a service on a chosen date.
<code>sp_UpdateScheduleStatus</code>	Changes schedule status (e.g. to Cancelled).

Service Details

Procedure Name	What It Does
<code>sp_AddServiceDetails</code>	Records work done, parts used, and cost for a service.
<code>sp_UpdateServiceStatus</code>	Updates status and auto-sets completion timestamp.

Complaint Management

Procedure Name	What It Does
----------------	--------------

sp_AddComplaint	Logs a new customer complaint.
sp_AssignComplaint	Assigns a complaint to a staff member.
sp_ResolveComplaint	Marks complaint as Resolved with resolution notes.
sp_UpdateComplaintStatus	Updates complaint status to any valid value.

Reports

Procedure Name	What It Does
sp_GetServiceReport	Returns all service records filtered by date range.
sp_GetCustomerHistory	Full service history for one customer.
sp_GetVehicleMaintenanceRecords	All maintenance records for one vehicle.

6 Functions

Functions are used to **calculate and return a single value**. Unlike stored procedures, they can be used directly inside SELECT queries, making them very flexible for reports and validation.

Function Name	Input	Returns	Purpose
<code>fn_GetVehicleTotalCost</code>	VehicleID	DECIMAL	Total cost of all completed services for a vehicle
<code>fn_GetCustomerServiceCount</code>	CustomerID	INT	Number of times a customer has used the service
<code>fn_HasActiveSchedule</code>	VehicleID	BIT (0/1)	Returns 1 if vehicle has a current booking
<code>fn_GetVehicleLabel</code>	VehicleID	VARCHAR	Returns a readable label: 'Toyota Corolla 2020 [RIW-1234]'
<code>fn_GetOpenComplaintCount</code>	CustomerID	INT	Number of unresolved complaints for a customer

Usage Example:

```
SELECT dbo.fn_GetVehicleTotalCost(1) AS TotalSpent;
SELECT dbo.fn_HasActiveSchedule(3) AS IsBooked;
```

7 Triggers

Triggers are **automatic actions** that the database runs by itself whenever data is inserted, updated, or deleted. No manual call is needed — they fire automatically behind the scenes.

Trigger Name	Table	When It Fires	What It Does
<code>trg_ServiceDetails_Insert</code>	ServiceDetails	AFTER INSERT	Sets the linked ServiceSchedule status to 'In Progress' automatically. Also writes to AuditLog.
<code>trg_ServiceDetails_Update</code>	ServiceDetails	AFTER UPDATE	If a service is marked Completed, the linked schedule is also set to Completed. Logs the change.
<code>trg_Vehicles_Audit</code>	Vehicles	AFTER INSERT / UPDATE / DELETE	Writes a full audit record every time a vehicle record is created, changed, or removed.
<code>trg_Customers_Audit</code>	Customers	AFTER INSERT / UPDATE / DELETE	Tracks all changes to customer records — name, phone, address — in the AuditLog.
<code>trg_Complaints_Audit</code>	Complaints	AFTER INSERT / UPDATE	Logs every new complaint and every status change for full complaint history tracking.

8 Business Logic Summary

This section explains the key business rules enforced by the database design in plain, simple language.

Login & Access

Only users in the Users table can log in. Role must be 'Admin' or 'Staff'. Admins have full access; Staff handle day-to-day operations. Deleted users are deactivated (IsActive=0), not removed.

Customer → Vehicle Link

Every vehicle must belong to a customer. You cannot add a vehicle without a valid CustomerID. Deleting a customer is blocked if they still own vehicles.

Service Booking Flow

A service is first scheduled (ServiceSchedule). When a technician begins work, ServiceDetails are added — this automatically moves the schedule to 'In Progress'. When work finishes, marking ServiceDetails as Completed also closes the schedule automatically.

Cost Tracking

Service costs are stored per service in ServiceDetails. The function fn_GetVehicleTotalCost() sums all completed service costs for any vehicle instantly.

Complaint Lifecycle

Complaints start as 'Open', get assigned to staff ('In Progress'), and are resolved with notes. ResolvedAt timestamp is auto-recorded. Open complaint counts are tracked per customer via fn_GetOpenComplaintCount().

Data Safety & Constraints

All tables use NOT NULL on essential fields. UNIQUE constraints prevent duplicate usernames, license plates, and VINs. CHECK constraints enforce valid status values so no invalid data can ever enter the system.

Full Audit Trail

Every important change — vehicle edits, customer updates, complaint status changes, service completions — is automatically logged in AuditLog by triggers. This provides a complete history of who changed what and when, without any extra effort from developers.

End of Database Design Document — Vehicle Service & Maintenance System — 2026